Mid-Level PHP (Laravel + Livewire + Alpine.js) — Project Test

Time Limit: 24 hours

Task Overview

Build a Purchase Entry Module and migrate legacy data and files into a clean structure.

Part 1 — Setup

Use Laravel (latest stable), install Livewire, and use Alpine.js.

Part 2 — Database

Create tables: items (id, name), brands (id, name), purchases (id, total, timestamps), purchase_items (id, purchase_id, item_id, brand_id, qty, price). Define relationships properly.

Part 3 — Purchase Form

Build a Livewire component with dynamic rows. Each row includes item, brand, quantity, and price. Use Alpine.js with Livewire entangle for reactivity. Prevent duplicate item+brand combinations. Show validation instantly and calculate total dynamically.

Part 4 — Role-Based Permissions

Implement role-based access control with at least two roles: Admin and User.

- Admin: Can create, edit, delete purchases and run migration.

- User: Can only view purchases.

Ensure routes, Livewire components, and actions are protected based on roles.

Part 5 — Legacy Data Migration

Given legacy data, map item_name and brand_name to respective tables, create records if missing, and insert into normalized tables. Ensure no duplicates and script is idempotent.

```
$legacyPurchases = [
   [
```

```
        'item_name' => 'Sugar',
        'brand_name' => 'ABC',
        'qty' => 10,
        'price' => 100,
    ]
];
```

Part 6 — Debugging Task (PHP MySQLi)

Fix the following buggy legacy code. Identify issues and provide a corrected version:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "test");

$id = $_GET['id'];

$query = "SELECT * FROM users WHERE id = $id";
$result = mysqli_query($conn, $query);

if($result){
    while($row = mysqli_fetch_assoc($result)){
        echo $row['name'];
    }
}else{
    echo "Query failed";
}

mysqli_close($conn);
?>
```

Expected: Secure, error-handled, and optimized version (e.g., prepared statements, validation, error handling).

Submission Requirements

Provide a Git repository with a README including setup steps, how to run migration script, and assumptions.

Important Rules

Do not hardcode IDs. Avoid heavy logic in Blade. Use proper validation. Keep code clean and structured.

Bonus (Optional)

Use transactions, debounce validation, and maintain clean UI/UX.